

Lab 02 – triedy, rekordy, rozhrania, enum, tuples

Ciel': precvičiť užitočné mechanizmy C#: indexery, voliteľné a pomenované parametre, lokálne a generické funkcie, default členov rozhraní, [Flags] alternatívy, record s dekonštrukciou a with, explicitnú implementáciu rozhraní a rozdiely medzi readonly struct a record struct.

Pokyny:

- Každá sekcia obsahuje presný popis toho, čo máte **dopísať** / **vytvoriť** / **overiť**.
- Kódové bunky obsahujú *starter* s TODO.
- Pod nimi sú malé **self-checky**, ktoré po správnom vyriešení prejdú.

1) Vlastná kolekcia s indexerom a spätným vyhľadávaním

Úloha:

Upravte triedu TagList, ktorá:

- interne drží List<string>,
- umožňuje prístup cez indexer this[int i] (get/set),
- umožňuje vyhľadanie indexu položky cez indexer this[string value] (iba get, vráťte -1 ak sa nenašlo),
- poskytuje metódy Add(string) a Contains(string).

Čo máte urobiť: doplniť indexery a metódy, následne **overiť** self-check.

```

public class TagList
{
    private readonly List<string> _data = new();

    // TODO: indexer pre int (get/set)
    public string this[int index]
    {
        get => _data[index];
        set => _data[index] = value;
    }

    // TODO: indexer pre string (iba get) → vráťte index, alebo -1\
    public int this[string value]
    {
        get => _data.IndexOf(value);
    }

    public void Add(string value) => _data.Add(value);
    public bool Contains(string value) => _data.Contains(value);
}

// self-check
var tags = new TagList();
tags.Add("csharp");
tags.Add("dotnet");
tags.Add("oop");
Console.WriteLine(tags[1]);           // očakávajte "dotnet"
Console.WriteLine(tags["oop"]);       // očakávajte 2
tags[0] = "C#";
Console.WriteLine(tags[0]);           // očakávajte "C#"

```

dotnet

2) Voliteľné parametre a pomenované argumenty –

FormatName

Úloha:

Implementujte metódu `FormatName`, ktorá skladá meno nasledovne:

`titleBefore FirstName MiddleName LastName, titleAfter`

- `titleBefore`, `middleName`, `titleAfter` sú voliteľné (`null` znamená nepoužiť),
- výstup nemá obsahovať nadbytočné medzery ani čiarky.

Čo máte urobiť: implementovať **celú metódu**, potom **overiť** self-check.

csharp

```
using System;
using System.Collections.Generic;
using System.Text;
Console.OutputEncoding = Encoding.UTF8;

string FormatName(
    string firstName,
    string lastName,
    string? titleBefore = null,
    string? middleName = null,
    string? titleAfter = null
)
{
    var parts = new List<string>();

    if (!string.IsNullOrEmpty(titleBefore))
        parts.Add(titleBefore.Trim());
```

```

parts.Add(firstName);
parts.Add(lastName);

// middleName patrí medzi krstné meno a priezvisko
if (!string.IsNullOrEmpty(middleName))
    parts.Insert(parts.Count - 1, middleName.Trim());

string core = string.Join(" ", parts);

if (!string.IsNullOrEmpty(titleAfter))
    core += $", {titleAfter.Trim()}";

return core;
}

// self-check
display(FormatName("Ján", "Novák")); // "Ján Novák"
Console.WriteLine(FormatName("Ján", "Novák", titleBefore: "Ing."));
Console.WriteLine(FormatName("Ján", "Novák", titleAfter: "ArtD."));
Console.WriteLine(FormatName("Ján", "Novák", middleName: "Adam"));
Console.WriteLine(FormatName("Ján", "Novák", titleBefore: "Bc.", titleAfter: "PhD"));

```

Ján Novák

Ing. Ján Novák

Ján Novák, ArtD.

Ján Adam Novák

Bc. Ján Novák, PhD.

3) Lokálna funkcia – normalizácia vstupu

Úloha:

V metóde `Process` vytvorte **lokálnu funkciu** `Normalize(string)` (`trim` + `ToLowerInvariant`).

Funkcia `Process` má prejsť slová, normalizovať ich a vypísať.

Čo máte urobiť: doplniť lokálnu funkciu a použitie, potom **overiť** self-check.

csharp

```
string Normalize(string value) =>
    value.Trim().ToLowerInvariant();
void Process(IEnumerable<string> words)
{
    // TODO: implementujte funkciu na normalizáciu reťazcov

    foreach (var w in words)
        Console.WriteLine(Normalize(w));
}

// self-check
Process(new [] { "  CSharp ", "  DOTNET", "  OOP  " }); // "csharp", "dotnet", "oop"
```

csharp

dotnet

4) Generická metóda – MinMax<T> s návratom tuple

Úloha:

Upravte generickú metódu `MinMax<T>(IEnumerable<T> items)` s obmedzením `where T : IComparable<T>`, aby vrátila n-ticu (`T min`, `T max`).

- Otestujte na `int` a `string` (lexikograficky).

Čo máte urobiť: upraviť metódu, potom **overiť** self-check.

csharp

```
(T min, T max) MinMax<T>(IEnumerable<T> items) where T : IComparable<T>
{
    using var e = items.GetEnumerator();
    if (!e.MoveNext())
        throw new InvalidOperationException("Sequence contains no elements");

    T min = e.Current;
    T max = e.Current;

    while (e.MoveNext())
    {
        T current = e.Current;

        if (current.CompareTo(min) < 0)
            min = current;

        if (current.CompareTo(max) > 0)
            max = current;
    }
}
```

```

        return (min, max);
    }

    // self-check
    var a = new[] { 5, 2, 9, 1, 4 };
    var (mi, ma) = MinMax(a);
    Console.WriteLine($"{mi} .. {ma}"); // 1 .. 9

    var b = new[] { "pear", "apple", "orange" };
    var (mis, mas) = MinMax(b);
    Console.WriteLine($"{mis} .. {mas}"); // apple .. pear

```

1 .. 9

apple .. pear

5) Rozhrania + tuple – IParser<T> s návratom (bool ok, T value)

Úloha:

Navrhňte rozhranie IParser<T> s metódou Parse(string s), ktorá vráti (bool ok, T value).

Implementujte aspoň dve verzie tried, ktoré implementujú toto rozhranie:

- IntParser (int.TryParse),
- DateParser (napr. DateTime.TryParse).

Čo máte urobiť: vytvoriť rozhranie, implementácie a **overiť** self-check.

```

using System;

public interface IParser<T>
{
    (bool ok, T value) Parse(string s);
}

public class IntParser : IParser<int>
{
    public (bool ok, int value) Parse(string s) ⇒
        (int.TryParse(s, out var v), v);
}

public class DateParser : IParser<DateTime>
{
    public (bool ok, DateTime value) Parse(string s) ⇒
        (DateTime.TryParse(s, out var v), v);
}

// self-check
IParser<int> pi = new IntParser();
IParser<DateTime> pd = new DateParser();
Console.WriteLine(pi.Parse("123"));           // (True, 123)
Console.WriteLine(pd.Parse("2025-10-08"));    // (True, 8. 10. 2025 0:00:00)
Console.WriteLine(pi.Parse("xxx"));           // (False, 0)

```

(True, 123)

(True, 10/8/2025 12:00:00 AM)

(False, 0)

6) Enum

6A) Enum + Tuple + switch

Úloha:

Definujte metódu `NextDay(Day d)`, ktorá vráti `(Day next, bool isWeekend)`, pričom v `next` bude nasledujúca hodnota enumu a v `isWeekend` bude hodnota, či táto nasledujúca hodnota je víkend (t. j. sobota alebo nedeľa).

Čo máte urobiť: definovať metódu `NextDay`, **overiť** self-check.

csharp

```
using System;

public enum Day { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday }

public static class DayHelper
{
    public static (Day next, bool isWeekend) NextDay(Day current)
    {
        int count = Enum.GetValues<Day>().Length; // 7
        Day next = (Day)(((int)current + 1) % count);
        bool isWeekend = next is Day.Saturday or Day.Sunday;
        return (next, isWeekend);
    }
}

// self-check
var (next, weekend) = DayHelper.NextDay(Day.Friday);
Console.WriteLine($"{next} - víkend? {weekend}"); // Saturday - víkend? True
```

Saturday – víkend? True

6B) [Flags]

Úloha:

Definujte [Flags] enum `TextStyle` (None, Bold, Italic, Underline, Monospace).

Implementujte metódu `Apply(string text, TextStyle style)`, ktorá obalí text značkami:

Bold $\Rightarrow$ `[B]text[/B]`, Italic $\Rightarrow$ `[I]text[/I]`, Underline $\Rightarrow$ `[U]text[/U]`, Monospace $\Rightarrow$ `[M]text[/M]`.

Čo máte urobiť: definovať enum, doplniť `Apply`, **overiť** self-check.

csharp

```
using System;

[Flags]
public enum TextStyle
{
    None = 0,
    Bold = 1 << 0,      // 1
    Italic = 1 << 1,     // 2
    Monospace = 1 << 2,  // 4
    Underline = 1 << 3   // 8
}

string Apply(string text, TextStyle style)
{
    string result = text;

    // 1. Najprv vnútorné vrstvy (Italic, Monospace)
    if ((style & TextStyle.Italic) == TextStyle.Italic)
        result = $"[I]{result}[/I]";
    if ((style & TextStyle.Monospace) == TextStyle.Monospace)
        result = $"[M]{result}[/M]";

    // 2. Potom vonkajšie vrstvy (Bold, Underline)
    if ((style & TextStyle.Bold) == TextStyle.Bold)
        result = $"[B]{result}[/B]";
    if ((style & TextStyle.Underline) == TextStyle.Underline)
        result = $"[U]{result}[/U]";
}
```

```
        return result;
    }

    // self-check
    var s = Apply("CSharp", TextStyle.Bold | TextStyle.Italic);
    Console.WriteLine(s); // [I][B]CSharp[/B][I]
    var t = Apply("code", TextStyle.Monospace | TextStyle.Underline);
    Console.WriteLine(t); // [U][M]code[/M][U]
```

```
[B][I]CSharp[/I][B]
```

```
[U][M]code[/M][U]
```

7) record

7A) Základný record + dekonštrukcia

Vytvorte record `PersonRecord(string FirstName, string LastName, int Age)` a pridajte `Deconstruct(out string f, out string l)`.

csharp

```
// TODO: definujte záznam PersonRecord s vlastnosťami FirstName, LastName, Age
public record PersonRecord(string FirstName, string LastName, int Age)
{
    public void Deconstruct(out string firstName, out string lastName)
    {
        firstName = FirstName;
        lastName = LastName;
    }
}

var p1 = new PersonRecord("Anna", "Nováková", 30);
```

```
var (f, l) = p1;
Console.WriteLine($"{f} {l}"); // Anna Nováková
```

Anna Nováková

7B) with a vnorený record

Preskúmajte record `Address(string City, string Street)` a record `PersonWithAddress(string FirstName, string LastName, int Age, Address Address)`.

Vytvorte kópiu osoby s upravenou adresou pomocou `with`.

csharp

```
public record Address(string City, string Street);
public record PersonWithAddress(string FirstName, string LastName, int Age, Address Address)

var pa1 = new PersonWithAddress("Ján", "Kováč", 25, new Address("ZA", "Hlavná"));
// TODO: vytvorte kópiu osoby s upravenou adresou pomocou `with`
var pa2 = pa1 with
{
    Address = pa1.Address with { Street = "Bočná 2" };
};
Console.WriteLine(pa2); // PersonWithAddress { FirstName = Ján, LastName = Kováč, Age = 25, Address = Address { City = ZA, Street = Bočná 2 } }
```

```
PersonWithAddress { FirstName = Ján, LastName = Kováč, Age = 25, Address =
Address { City = ZA, Street = Bočná 2 } }
```

8) Explicitná implementácia rozhrania

8A) Rozlíšenie mien

Vytvorte triedu `SecretDocument` implementujúcu rozhranie `IPrintable`, pričom pridajte verejnú implicitnú inštančnú metódu `Print()`, ktorá vypíše "Public print" a okrem toho pridajte aj druhú explicitnú metódu `IPrintable.Print()`, ktorá vypíše "Secret print".

csharp

```
using System;

public interface IPrintable { void Print(); }

public class SecretDocument : IPrintable
{
    public void Print()
    {
        Console.WriteLine("Public print");
    }

    void IPrintable.Print()
    {
        Console.WriteLine("Secret print");
    }
}

var doc = new SecretDocument();
doc.Print(); // Public print
((IPrintable)doc).Print(); // Secret print
```

Public print

Secret print

8B) Kolekcia

CsvCollection s interným List<string>: explicitne IEnumerable (negerické) a implicitne IEnumerable<string>.

csharp

```
public class CsvCollection : IEnumerable<string>, System.Collections.IEnumerab
{
    private readonly List<string> _items = new();
    public void Add(string line) => _items.Add(line);

    public IEnumerator<string> GetEnumerator() => _items.GetEnumerator();
    // TODO: Pridajte implementáciu negenerického GetEnumerator

    IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
}

var csv = new CsvCollection();
csv.Add("a,1");
csv.Add("b,2");
foreach (var line in csv)
    Console.WriteLine(line);
// a,1
// b,2
```

a,1

b,2

9) Porovnajete správanie pri priradení pre class a struct.

var r1 = new RefPoint(1, 1); // r1 ukazuje na objekt v halde

var r2 = r1; // r2 ukazuje NA TEN ISTÝ objekt

r1.X *= 10; // mení sa objekt, na ktorý ukazujú obe premenné

// r1.X = 10, r2.X = 10

csharp

```
public class RefPoint
{
    public int X { get; set; }
    public int Y { get; set; }
    public RefPoint(int x, int y) { X = x; Y = y; }
}

public struct ValuePoint
{
    public int X { get; set; }
    public int Y { get; set; }
    public ValuePoint(int x, int y) { X = x; Y = y; }
}

var r1 = new RefPoint(1, 1);
var r2 = r1;
r1.X *= 10;
Console.WriteLine($"RefPoint: r1.X={r1.X}, r2.X={r2.X}"); // RefPoint: r1.X=10

var v1 = new ValuePoint(1, 1);
var v2 = v1;
v1.X *= 10;
Console.WriteLine($"ValuePoint: v1.X={v1.X}, v2.X={v2.X}"); // ValuePoint: v1.X=10
```

RefPoint: r1.X=10, r2.X=10

ValuePoint: v1.X=10, v2.X=1

10) readonly struct a record struct – porovnanie

10A) readonly struct Vector2:

```
readonly struct Vector2(double X, double Y) { Length();
```

HashSet<T> používa EqualityComparer<T>.Default, ktorý pre struct volá
ValueType.Equals(object). Ten:

porovnáva všetky inštancové polia pomocou reflexie,

ak sú polia rovnaké, vráti true.

Takže teoreticky by HashSet mal vrátiť 1. Ale – ValueType.Equals je pomalé a v praxi sa neodporúča spoliehať na neho. Navyše, GetHashCode pre ValueType je tiež implementovaný cez reflexiu prvého poľa – a to je nedostatočné pre správne hashovanie.

Skutočný výsledok

V .NET 10 s C# 12 je správanie takéto:

Kód Výsledok

v1.Length() 5

v2.Length() 5

hs.Count po dvoch Add(new Vector2(1,2)) 1 (v modernom .NET) alebo 2 (v závislosti od verzie a kontextu)

V praxi sa často stane, že HashSet vráti 2, pretože ValueType.GetHashCode pre structy s viacerými poľami nie je spoľahlivý – môže vrátiť rôzne hash kódy pre dve hodnotovo rovnaké inštancie. Potom HashSet ich považuje za rôzne, aj keď Equals vráti true.

csharp

```
public readonly struct Vector2
{
    public double X { get; }
    public double Y { get; }
    public Vector2(double x, double y) { X = x; Y = y; }
    public double Length() => Math.Sqrt(X*X + Y*Y);
}

var v1 = new Vector2(3, 4);
var v2 = v1;
Console.WriteLine(v1.Length()); // 5
Console.WriteLine(v2.Length()); // 5

var hs = new HashSet<Vector2>();
hs.Add(new Vector2(1,2));
hs.Add(new Vector2(1,2));
Console.WriteLine(hs.Count); // 1
```

10B) record struct Point:

record struct Point(double X, double Y) – dekonštrukcia, rovnosť, HashSet<Point>.

csharp

```
public readonly record struct Point(double X, double Y);

var p = new Point(1, 2);
var (px, py) = (p.X, p.Y);
Console.WriteLine($"{px},{py}"); // 1,2

var hs = new HashSet<Point>();
hs.Add(new Point(1,2));
hs.Add(new Point(1,2));
Console.WriteLine(hs.Count); // 1
```

11) Primárny konštruktor (c# 12) – bankový účet

Úloha:

- Pridajte primárny konštruktor do triedy `class BankAccount` a vlastnosti `AccountId`, `Owner` iba na čítanie, `Balance` s `private set`.

csharp

```
public class BankAccount
{
    public string AccountId { get; }
    public string Owner { get; }
    public decimal Balance { get; private set; }

    public BankAccount(string accountId, string owner)
    {
        AccountId = accountId;
        Owner = owner;
        Balance = 0m;
    }

    public void Deposit(decimal amount)
    {
        if (amount ≤ 0) throw new ArgumentOutOfRangeException(nameof(amount))
        Balance += amount;
    }

    public void Withdraw(decimal amount)
    {
        if (amount ≤ 0) throw new ArgumentOutOfRangeException(nameof(amount))
        if (Balance < amount) throw new InvalidOperationException("Insufficient")
        Balance -= amount;
    }

    public override string ToString() ⇒ $"{AccountId} - {Owner}: {Balance:C}"
}

var acc = new BankAccount("SK00-123", "Ján");
acc.Deposit(100);
```

```
acc.Withdraw(40);  
Console.WriteLine(acc); // SK00-123 - Ján: 60,00 €
```

SK00-123 â€“ JÃ¡n: \$60.00

